



UNSW
SYDNEY

COMP4141

Theory of Computation

Lecture 4: Regular Expressions

Outline

Regular Expressions

From Regular Expressions to Finite Automata

From DFAs to Regular Expressions

End Matter

Regular Expression Definition

Regular expressions are defined relative to some alphabet Σ .

This is a recursive definition of the structure of regular expressions.

The structure of regular expressions is the basis for other recursive definitions and induction proofs. Every recursive definition and proof has to handle these six cases:

- a is a regular expression, if $a \in \Sigma$.
- $\emptyset$ is a regular expression.
- ϵ is a regular expression.
- $R_1 \cup R_2$ is a regular expression if R_1 and R_2 are.
- $R_1 \cdot R_2$ is a regular expression if R_1 and R_2 are.
- R^* is a regular expression if R is.

Parentheses can be added in the obvious places to override precedence: $*$ has the highest precedence, followed by $\cdot$, and finally $\cup$ which has the lowest precedence, so $a \cup b \cdot c^* = a \cup (b \cdot (c^*))$.

The first three definitions can be considered base cases while the last three are inductive.

Concise Regular Expression Definition

Using so called *EBNF* (for *extended Backus-Naur form*) the syntax of regular expressions RE_Σ over Σ can be defined by:

$$\begin{aligned}\text{RE}_\Sigma \ni R &::= a \mid \emptyset \mid \epsilon \mid R \cup R \mid R \cdot R \mid R^* \\ \Sigma \ni a &::= \dots\end{aligned}$$

(The second line is for enumerating the letters of the alphabet Σ .)

Language of a Regular Expression

Let R be a regular expression. The language $L(R)$ of R is defined recursively on the structure of R .

- Case R is a for some $a \in \Sigma$: $L(a) = \{a\}$
- Case R is $\emptyset$: $L(\emptyset) = \emptyset$
- Case R is ϵ : $L(\epsilon) = \{\epsilon\}$
- Case R is $R_1 \cup R_2$: $L(R_1 \cup R_2) = L(R_1) \cup L(R_2)$
- Case R is $R_1 \cdot R_2$: $L(R_1 \cdot R_2) = L(R_1)L(R_2)$
- Case R is R_1^* : $L(R_1^*) = (L(R_1))^*$

NB

L relates each syntactic object to a semantic object, whence we also call it a semantics for reg. exps.

$$L : RE_{\Sigma} \longrightarrow 2^{\Sigma^*}$$

Regular Expression Questions

For the first 6 points, let $\Sigma = \{0, 1\}$.

- 1 How do we write Σ = “any one symbol” using existing operators?
- 2 What is $L(0^*1^*)$?
- 3 How do we write “all strings ending in 11”?
- 4 What is $L((0^*1^*)^*)$?
- 5 how do we write “at least one 0”?
- 6 And how do we write “at least one 0 and at least one 1”?
- 7 How do we write $R^+ = R^1 \cup R^2 \cup \dots$ using existing operators?
- 8 $R?$ means “an optional R ”. How do we write it?

Answers

- ① Σ can be defined by $0 \cup 1$; we use the abbreviation Σ below
- ② any number of 0s followed by any number of 1s
- ③ Σ^*11
- ④ the same as Σ^* i.e. all strings over the alphabet $\{0, 1\}$
- ⑤ $\Sigma^*0\Sigma^*$
- ⑥ $\Sigma^*(01 \cup 10)\Sigma^*$
- ⑦ RR^*
- ⑧ $\epsilon \cup R$

Some simple proof exercises

- 1 Suppose that R is a regular expression formed without use of $*$. Show that $L(R)$ is finite.
- 2 Suppose that R is a regular expression that contains a use of $*$ but does not contain a use of $\emptyset$ or ϵ . Show that $L(R)$ is infinite.

Outline

Regular Expressions

From Regular Expressions to Finite Automata

From DFAs to Regular Expressions

End Matter

From Regular Expressions to Finite Automata

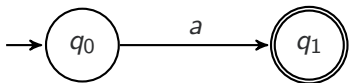
Theorem

For every regular expression R , there is an ϵ -NFA N_R such that $L(R) = L(N_R)$.

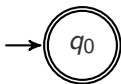
Proof.

by induction on the structure of R . For each of the three base cases and each of the three inductive cases we define an ϵ -NFA which recognises $L(R)$. □

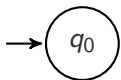
From Regular Expressions to Finite Automata cont.



a



ϵ



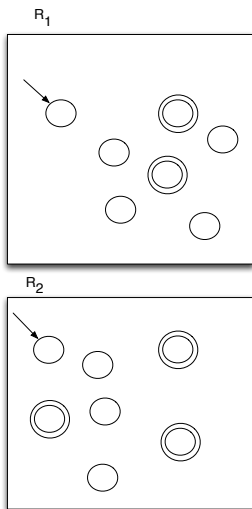
$\emptyset$

From Regular Expressions to Finite Automata cont.

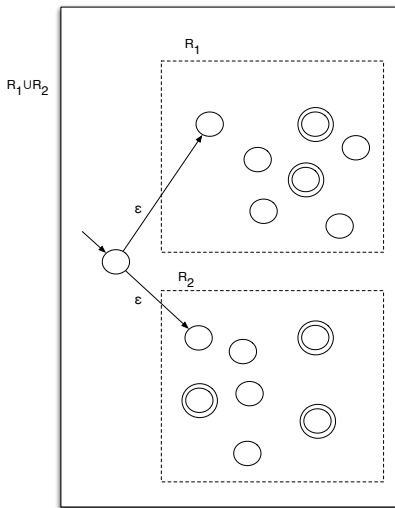
Base Cases (formally):

- $N_a = (\{q_0, q_1\}, \Sigma, \delta_a, q_0, \{q_1\})$ where $\delta_a(q_0, a) = \{q_1\}$ and $\delta_a(q, s) = \emptyset$, for all $(q, s) \neq (q_0, a)$.
- $N_\emptyset = (\{q_0\}, \Sigma, \delta_\emptyset, q_0, \emptyset)$ where $\delta_\emptyset(q_0, s) = \emptyset$ for all $s \in \Sigma \cup \{\epsilon\}$.
- $N_\epsilon = (\{q_0\}, \Sigma, \delta_\emptyset, q_0, \{q_0\})$.

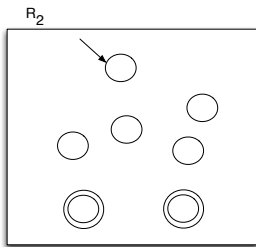
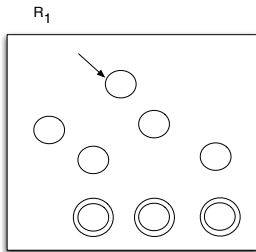
From Regular Expressions to Finite Automata (Union)



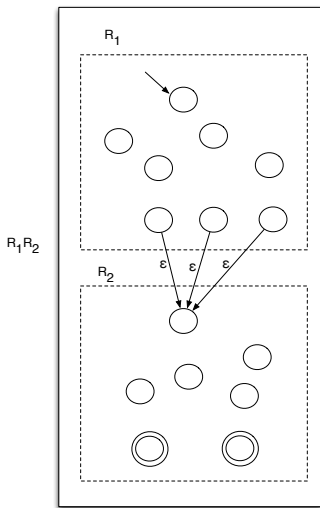
From Regular Expressions to Finite Automata (Union)



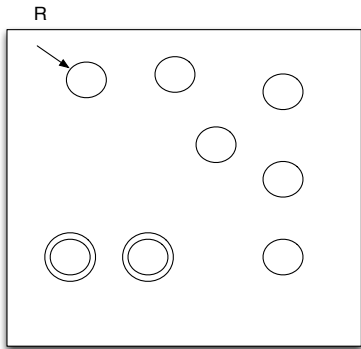
From Regular Expressions to Finite Automata (Concatenation)



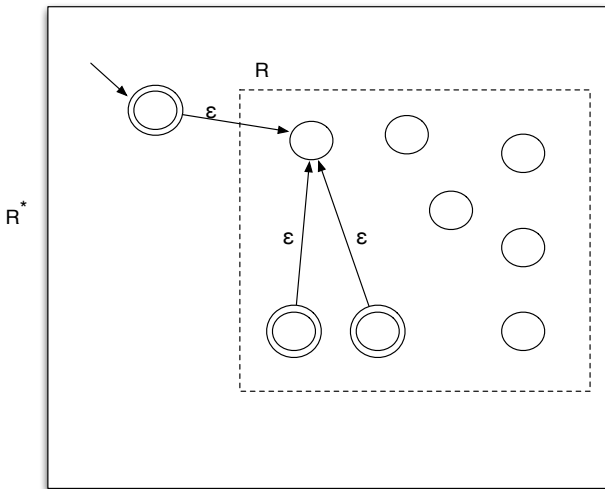
From Regular Expressions to Finite Automata (Concatenation)



From Regular Expressions to Finite Automata (Kleene Star)



From Regular Expressions to Finite Automata (Kleene Star)



From Regular Expressions to Finite Automata cont.

Inductive Cases (formally):

- For $i \in \{1, 2\}$ let $N_i = (Q_i, \Sigma, \delta_i, q_0^{(i)}, F_i)$ be an ϵ -NFA satisfying $L(N_i) = L(R_i)$ according to the induction hypothesis. Moreover we assume w.l.o.g. that the two sets of states, Q_1 and Q_2 are disjoint and do not contain q_0 .
$$N_{R_1 \cup R_2} = (\{q_0\} \cup Q_1 \cup Q_2, \Sigma, \delta_1 \cup \delta_2 \cup \left\{ (q_0, \epsilon) \mapsto q_0^{(i)} \mid 1 \leq i \leq 2 \right\}, q_0, F_1 \cup F_2).$$
- With the same N_i as in the previous case, define
$$N_{R_1 \cdot R_2} = (Q_1 \cup Q_2, \Sigma, \delta_1 \cup \delta_2 \cup \left\{ (q, \epsilon) \mapsto \delta_1(q, \epsilon) \cup \{q_0^{(2)}\} \mid q \in F_1 \right\}, q_0^{(1)}, F_2).$$
- Let $N = (Q, \Sigma, \delta, q'_0, F)$ be an ϵ -NFA satisfying $L(N) = L(R_1)$ according to the induction hypothesis. W.l.o.g. assume that $q_0 \notin Q$. Define
$$N_{R_1^*} = (\{q_0\} \cup Q, \Sigma, \delta \cup \{ (q, \epsilon) \mapsto q'_0 \mid q \in F \cup \{q_0\} \}, q_0, F).$$

Implications

We already know that ϵ -NFAs, NFAs, and DFAs have equal expressive power.

The latest theorem shows that FA are at least as expressive as regular expressions.

Next: Can we do anything with a FA that we cannot do with a regular expression?

From FA to Regular Expressions

Theorem

For every finite automaton (any type) A , there is a regular expression R such that $L(R) = L(A)$.

Outline

Regular Expressions

From Regular Expressions to Finite Automata

From DFAs to Regular Expressions

End Matter

FA $\rightarrow$ RE

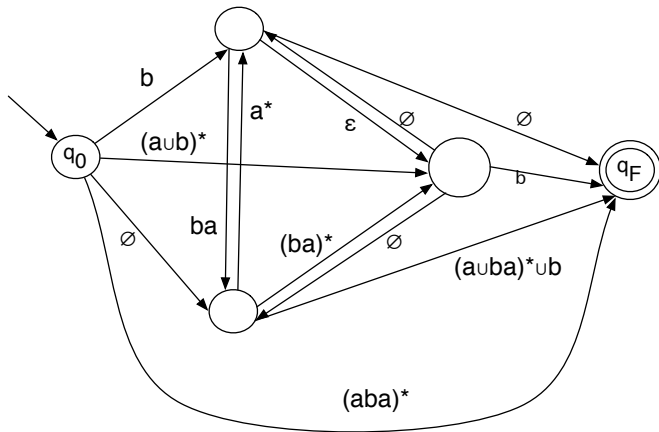
Let Σ be an alphabet. We write RE_{Σ} for the set of all regular expressions over Σ .

Roadmap: DFA $\rightarrow$ GNFA $\rightarrow$ RE_{Σ}

where a GNFA is an NFA with

- regular expressions instead of symbols as labels on transitions,
- a unique final state,
- and a full transition relation, except that there are no transitions either (a) into the start state or (b) out of the accept state.

GNFA example



Definition

A *generalised non-deterministic finite automaton (GNFA)* is a 5-tuple $(Q, \Sigma, \delta, q_0, q_F)$ where

- 1 Q is finite set of states,
- 2 Σ is the input alphabet,
- 3 $\delta : (Q \setminus \{q_F\}) \times (Q \setminus \{q_0\}) \longrightarrow \text{RE}_\Sigma$ is the transition function,
- 4 $q_0 \in Q$ is the start state, and
- 5 $q_F \in Q$ is the accept state.

Language of a GNFA

Definition

A GNFA accepts a string $w \in \Sigma^*$ if there exists a $k \in \mathbb{N}$, a sequence of states $q_1, \dots, q_k$, and a sequence of strings $w_1, \dots, w_k$ such that

- 1 $w = w_1 \dots w_k$
- 2 $w_i \in L(\delta(q_{i-1}, q_i))$, for $i \in \{1, \dots, k\}$
- 3 $q_k = q_F$

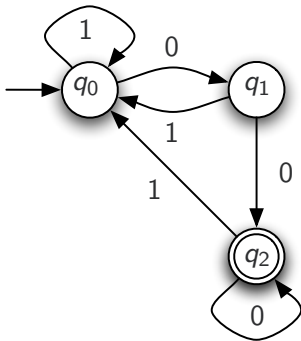
Note that this already implies that execution started at q_0 .

DFA $\longrightarrow$ GNFA

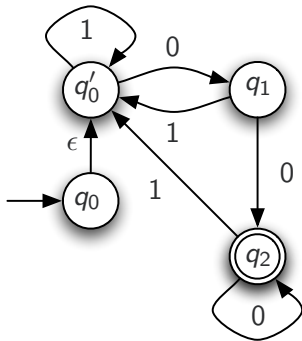
This is straightforward.

- Add a new start state, connect it via an ϵ transition to the original start state (which is no longer the initial state).
- Add a new accept state, connect all the original accept states via ϵ transition to it. (They are no longer accept states.)
- Between pairs (q, q') of original states, replace all existing transitions by a single one labeled with the (regular expression) union of the labels on the original transitions from q to q' .
- Introduce $\emptyset$ -labeled transitions where needed (e.g. from the new start state to all but the old start state).

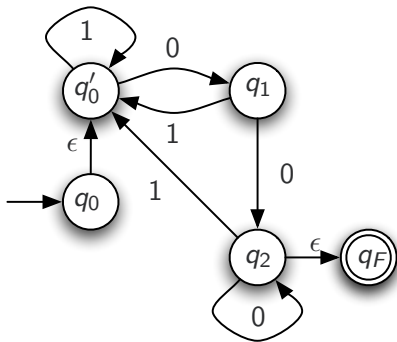
Example DFA $\longrightarrow$ GNFA



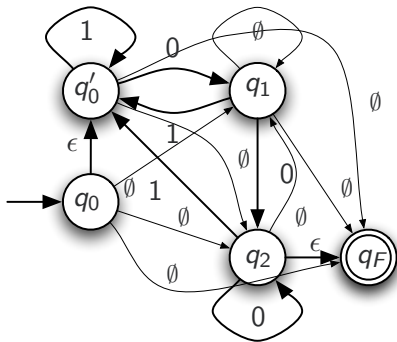
Example DFA $\longrightarrow$ GNFA



Example DFA $\longrightarrow$ GNFA



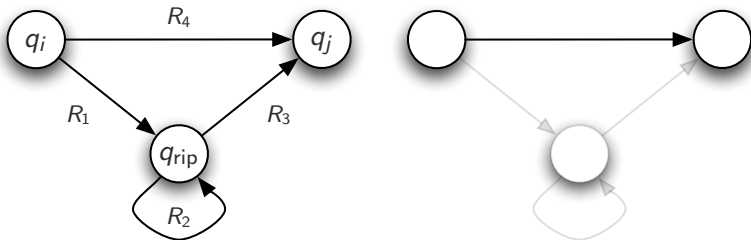
Example DFA $\longrightarrow$ GNFA



GNFA $\rightarrow$ RE $_{\Sigma}$

The gist of this construction is to eliminate all *inner* states of the GNFA (i.e. the states of the original DFA) one by one. When all of them are gone, only q_0 and q_F remain. The only remaining transition (from q_0 to q_F) is labeled with the regular expression we're after.

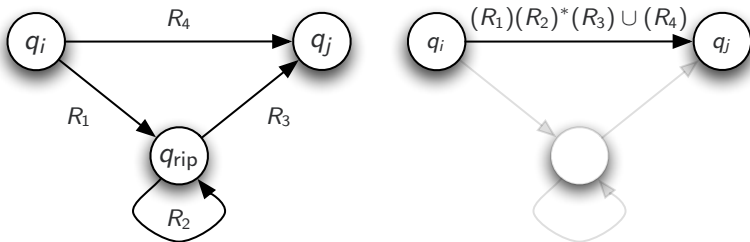
To eliminate an “inner” state q_{rip} , we need to augment the labels on every transition from $q_i \neq q_{rip}$ to $q_j \neq q_{rip}$:



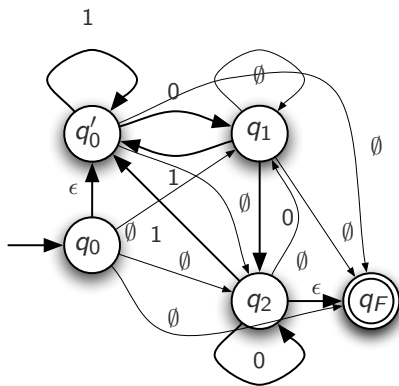
GNFA $\rightarrow$ RE $_{\Sigma}$

The gist of this construction is to eliminate all *inner* states of the GNFA (i.e. the states of the original DFA) one by one. When all of them are gone, only q_0 and q_F remain. The only remaining transition (from q_0 to q_F) is labeled with the regular expression we're after.

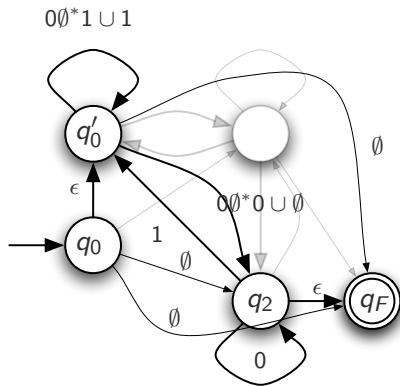
To eliminate an “inner” state q_{rip} , we need to augment the labels on every transition from $q_i \neq q_{rip}$ to $q_j \neq q_{rip}$:



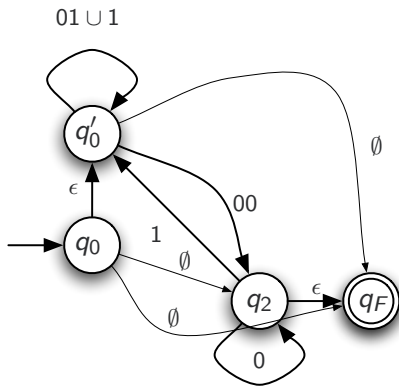
Example GNFA $\longrightarrow$ RE_Σ



Example GNFA $\longrightarrow$ RE_{Σ}

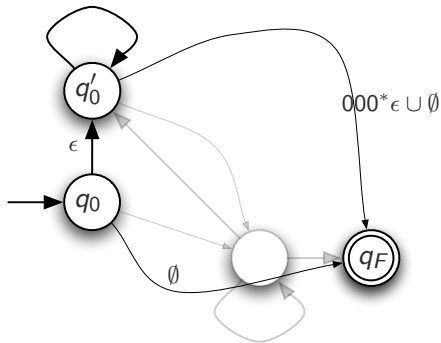


Example GNFA $\longrightarrow$ RE_Σ



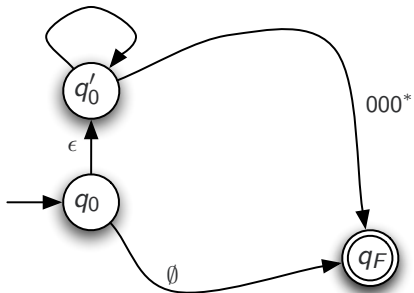
Example GNFA $\longrightarrow$ RE $_{\Sigma}$

$000^*1 \cup 01 \cup 1$

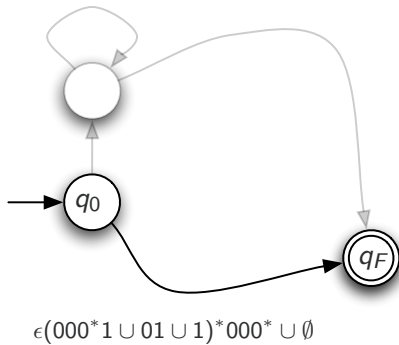


Example GNFA $\longrightarrow$ RE $_{\Sigma}$

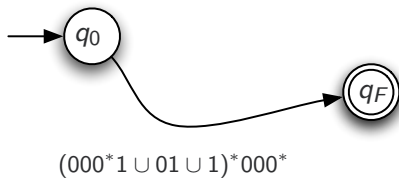
$000^*1 \cup 01 \cup 1$



Example GNFA $\longrightarrow$ RE $_{\Sigma}$



Example GNFA $\longrightarrow$ RE $_{\Sigma}$



Regular languages

Theorem (Kleene's theorem)

Let $L \subseteq \Sigma^$ be a language. The following are equivalent:*

- *L is accepted by a DFA*
- *L is accepted by an NFA*
- *L is accepted by a GNFA*
- *$L = L(R)$ for a regular expression R*

Outline

Regular Expressions

From Regular Expressions to Finite Automata

From DFAs to Regular Expressions

End Matter

Weekly Feedback

I would appreciate any comments/suggestions/requests you have on this week's lectures (and tutorials).



<https://forms.office.com/r/g3LgPASave>